

INTRODUCTION:

- database is a collection of related data

↙
general definition

↳ facts that can be recorded and that have implicit meaning

- A database represents some aspect of the real world, sometimes called **miniworld** or the **universe of discourse (UoD)**. Changes to the miniworld are reflected in the database.
- A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.
- A database is designed, built, and populated with data for a specific purpose
- a database can be of any size and complexity.

→ DBMS:

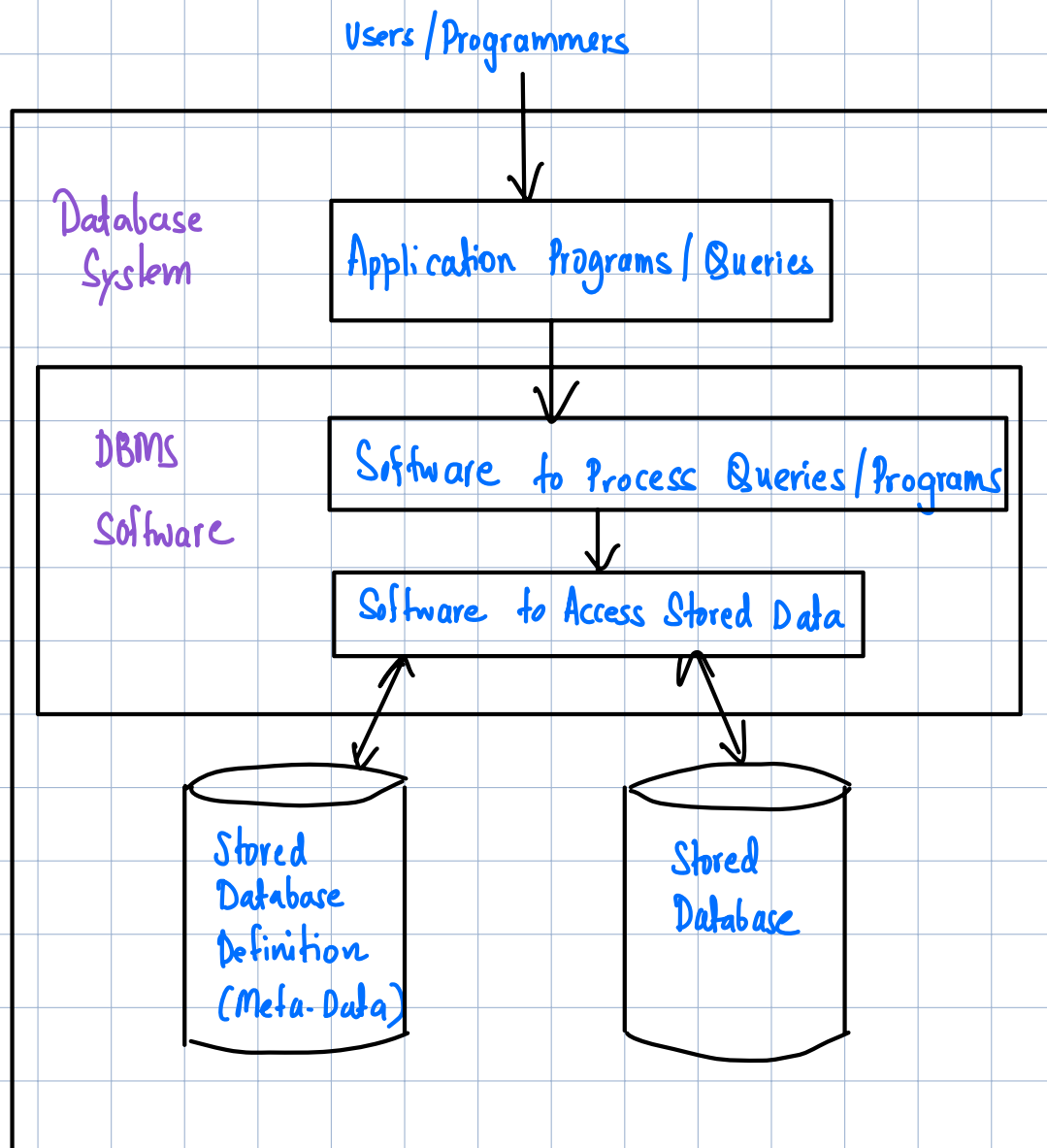
- DBMS is a **computerized** system that enables users to **create** and **maintain** a database.
- The DBMS is a general-purpose software system that facilitates the processes of **defining**, **constructing**, **manipulating**, and **sharing** databases among various users and applications
 - Defining → specifying **data types**, **structures** and **constraints** of the data.
 - ↳ stored as a **database catalog** or dictionary
 - ↳ meta-data
 - Constructing → process of storing the data.

- Manipulating → querying the database to retrieve specific data, updating the database to reflect changes.
- Sharing → allows multiple users and programs to access the database simultaneously.

* application program accesses the database by sending queries or requests for data to the DBMS.

* a transaction may cause some data to be read and some data to be written into the database.

- DBM also helps for protection and maintenance.



A simplified database system environment.

→ Process of designing DBMS:

- (i) Requirements specification and analysis
- (ii) conceptual design
- (iii) logical design
- (iv) Physical design

CHARACTERISTICS OF THE DATABASE APPROACH:

(i) Self-describing nature of a database system

- database system contains not only the database itself but also a complete definition or description of the database structure and constraints.
↳ catalog

(ii) Insulation b/w Programs and Data, and Data abstraction:

- the structure of data files is stored in DBMS catalog separately from the access programs. We call this property **program-data independence**
 - an operation (function or method)
 - ↓
interface (signature)
↓
operation name and datatypes of arguments
 - ↓
implementation (method)
- } **program-operation independence**

- the characteristic that allows program-data independence and program-operation independence is called data abstraction.

(iii) Support of Multiple Views of the Data:

- A view may be a subset of the database or it may contain virtual data that is derived from the database files but is not explicitly stored.

(iv) Sharing of Data and Multiuser Transaction Processing:

- allows multiple users to access the database at the same time.
- concurrency control
 - ↳ the isolation property ensures that each transaction appears to execute in isolation from other transactions, even though hundreds of hundreds may be executing concurrently.
 - ↳ the atomicity property ensures that either all the database operations in a transaction are executed or none are.

1.4 Actors on the Scene

For a small personal database, such as the list of addresses discussed in Section 1.1, one person typically defines, constructs, and manipulates the database, and there is no sharing. However, in large organizations, many people are involved in the design, use, and maintenance of a large database with hundreds or thousands of users. In this section we identify the people whose jobs involve the day-to-day use of a large database; we call them the *actors on the scene*. In Section 1.5 we consider people who may be called *workers behind the scene*—those who work to maintain the database system environment but who are not actively interested in the database contents as part of their daily job.

1.4.1 Database Administrators

In any organization where many people use the same resources, there is a need for a chief administrator to oversee and manage these resources. In a database environment, the primary resource is the database itself, and the secondary resource is the DBMS and related software. Administering these resources is the responsibility of the **database administrator (DBA)**. The DBA is responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources as needed. The DBA is accountable for problems such as security breaches and poor system response time. In large organizations, the DBA is assisted by a staff that carries out these functions.

1.4.2 Database Designers

Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data. These tasks are mostly undertaken before the database is actually implemented and populated with data. It is the responsibility of database designers to communicate with all prospective database users in order to understand their requirements and to create a design that meets these requirements. In many cases, the designers are on the staff of the DBA and may be assigned other staff responsibilities after the database design is completed. Database designers typically interact with each potential group of users and develop **views** of the database that meet the data and processing requirements of these groups. Each view is then analyzed and *integrated* with the views of other user groups. The final database design must be capable of supporting the requirements of all user groups.

1.4.3 End Users

End users are the people whose jobs require access to the database for querying, updating, and generating reports; the database primarily exists for their use. There are several categories of end users:

- **Casual end users** occasionally access the database, but they may need different information each time. They use a sophisticated database query interface

to specify their requests and are typically middle- or high-level managers or other occasional browsers.

- **Naive or parametric end users** make up a sizable portion of database end users. Their main job function revolves around constantly querying and updating the database, using standard types of queries and updates—called **canned transactions**—that have been carefully programmed and tested. Many of these tasks are now available as **mobile apps** for use with mobile devices. The tasks that such users perform are varied. A few examples are:
 - Bank customers and tellers check account balances and post withdrawals and deposits.
 - Reservation agents or customers for airlines, hotels, and car rental companies check availability for a given request and make reservations.
 - Employees at receiving stations for shipping companies enter package identifications via bar codes and descriptive information through buttons to update a central database of received and in-transit packages.
 - Social media users post and read items on social media Web sites.
- **Sophisticated end users** include engineers, scientists, business analysts, and others who thoroughly familiarize themselves with the facilities of the DBMS in order to implement their own applications to meet their complex requirements.
- **Standalone users** maintain personal databases by using ready-made program packages that provide easy-to-use menu-based or graphics-based interfaces. An example is the user of a financial software package that stores a variety of personal financial data.

A typical DBMS provides multiple facilities to access a database. Naive end users need to learn very little about the facilities provided by the DBMS; they simply have to understand the user interfaces of the mobile apps or standard transactions designed and implemented for their use. Casual users learn only a few facilities that they may use repeatedly. Sophisticated users try to learn most of the DBMS facilities in order to achieve their complex requirements. Standalone users typically become very proficient in using a specific software package.

1.4.4 System Analysts and Application Programmers (Software Engineers)

System analysts determine the requirements of end users, especially naive and parametric end users, and develop specifications for standard canned transactions that meet these requirements. **Application programmers** implement these specifications as programs; then they test, debug, document, and maintain these canned transactions. Such analysts and programmers—commonly referred to as **software developers** or **software engineers**—should be familiar with the full range of capabilities provided by the DBMS to accomplish their tasks.

1.5 Workers behind the Scene

In addition to those who design, use, and administer a database, others are associated with the design, development, and operation of the DBMS *software and system environment*. These persons are typically not interested in the database content itself. We call them the *workers behind the scene*, and they include the following categories:

- **DBMS system designers and implementers** design and implement the DBMS modules and interfaces as a software package. A DBMS is a very complex software system that consists of many components, or **modules**, including modules for implementing the catalog, query language processing, interface processing, accessing and buffering data, controlling concurrency, and handling data recovery and security. The DBMS must interface with other system software, such as the operating system and compilers for various programming languages.
- **Tool developers** design and implement **tools**—the software packages that facilitate database modeling and design, database system design, and improved performance. Tools are optional packages that are often purchased separately. They include packages for database design, performance monitoring, natural language or graphical interfaces, prototyping, simulation, and test data generation. In many cases, independent software vendors develop and market these tools.
- **Operators and maintenance personnel** (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system.

Although these categories of workers behind the scene are instrumental in making the database system available to end users, they typically do not use the database contents for their own purposes.

DATA MODELS, SCHEMAS, AND INSTANCES:

→ Data abstraction generally refers to the suppression of details of data organization and storage, and the highlighting of the essential features for an improved understanding of data.

⇒ DATA MODELS:

→ a **data model** - a collection of concepts that can be used to describe the structure of a database - provides the necessary means to achieve this abstraction.

datatypes, relationships, and constraints that apply to the

data.

- most data models also include a set of basic operations for specifying retrievals and updates on the database.
- also include dynamic aspect or behavior of database application

→ Categories of Data Models:

(i) High-Level(or) Conceptual data models:

- ↳ provide concepts that are close to the way many users perceive data.
- ↳ use entities, attributes and relationships.

(ii) Representational (or) implementation data models:

- most frequently used in commercial DBMS.
- eg: relational data model, network and hierarchical models
- representational data models represent data by using record structures and hence are sometimes called record-based data models.
- object data model are new family of higher-level implementation data models ↳ Object Data Management Group (ODMG)

(iii) Physical (or) low level data models:

- describes how data is stored as files in the computer by representing information such as record formats, record orderings.

and access paths.

↳ search structure that makes the search for particular database records efficient, such as indexing or hashing.

→ another class of data models is known as self-describing data models.

↳ combines the description of the data with the data values themselves.

⇒ SCHEMAS, INSTANCES, AND DATABASE STATE

→ in a data model, it is important to distinguish b/w the description of the database and the database itself.

→ the description of a database is called the database schema, which is specified during database design and is not expected to change frequently.

Figure 2.1
Schema diagram for
the database in
Figure 1.2.

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

- the data in the database at a particular moment in time is called database state or snapshot

↳ current set of occurrences or instances

The distinction between database schema and database state is very important. When we **define** a new database, we specify its database schema only to the DBMS. At this point, the corresponding database state is the *empty state* with no data. We get the *initial state* of the database when the database is first **populated** or **loaded** with the initial data. From then on, every time an update operation is applied to the database, we get another database state. At any point in time, the database has a *current state*.⁸ The DBMS is partly responsible for ensuring that every state of the database is a **valid state**—that is, a state that satisfies the structure and constraints specified in the schema. Hence, specifying a correct schema to the DBMS is extremely important and the schema must be designed with utmost care. The DBMS stores the descriptions of the schema constructs and constraints—also called the **meta-data**—in the DBMS catalog so that DBMS software can refer to the schema whenever it needs to. The schema is sometimes called the **intension**, and a database state is called an **extension** of the schema.

- changes made to schema is called **schema evolution**

THREE - SCHEMA ARCHITECTURE & DATA INDEPENDENCE:

⇒ The three - Schema Architecture:

→ the goal of the three-schema architecture, is to separate the user applications from the physical database.

→ there are 3 levels:

① internal level:

↳ has internal schema

↳ describes physical storage structure of the database.

↳ uses physical data model and describes the complete details of data storage and access paths of the database.

② Conceptual level:

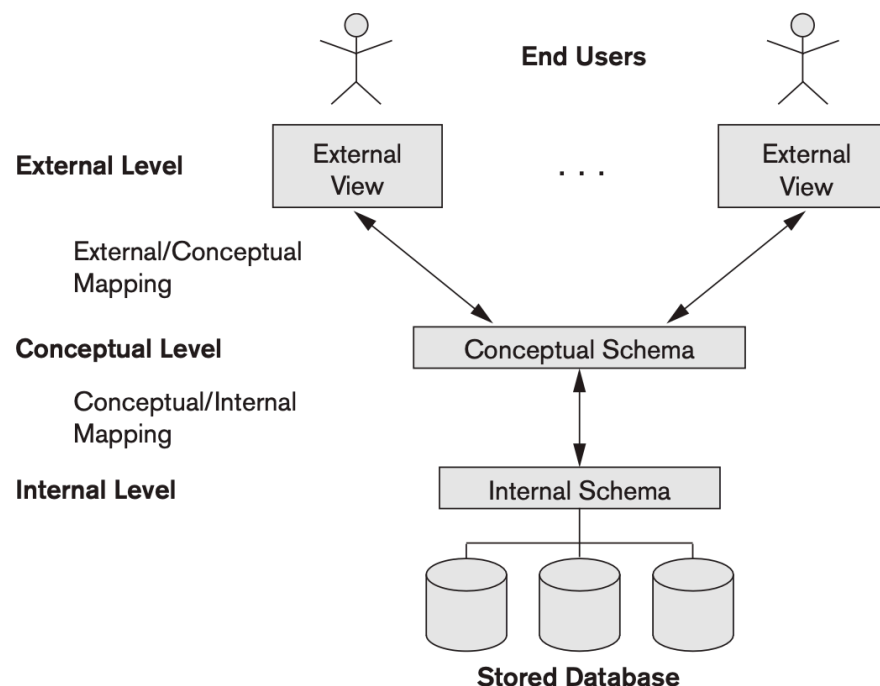
↳ has conceptual schema

- ↳ describes the structure of the whole database for a community of users.
- ↳ hides the details of physical structures and concentrates on describing entities, datatypes, relationships, user operations, and constraints.

③ The external or view level :

- ↳ includes a no. of external schemas or user views.
- ↳ each external schema describes the part of the database that a particular user group is interested in and hides the rest of the data base from that user group
- ↳ implemented using representational data model, possibly based on an external schema design in a high-level conceptual data model.

Figure 2.2
The three-schema architecture.



★ the three schemas are only descriptions of data, the actual data is stored at the physical level only.

In the three-schema architecture, each user group refers to its own external schema.

★ The processes of transforming requests and results b/w levels are called mappings.

⇒ Data Independence:

↳ capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

↳ 2 types of data independence:

① Logical data independence:

- capacity to change the conceptual schema without having to change external schemas or application programs.

- only the view definition and the mappings need to be changed in a DBMS that supports logical data independence.

- changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs.

② Physical data independence:

↳ capacity to change the internal schema without

having to change conceptual schema. Hence external schema need not be changed as well.
→ changes to the internal schema may be needed because some physical files were reorganized.

Generally, physical data independence exists in most databases and file environments where physical details, such as the exact location of data on disk, and hardware details of storage encoding, placement, compression, splitting, merging of records, and so on are hidden from the user. Applications remain unaware of these details. On the other hand, logical data independence is harder to achieve because it allows structural and constraint changes without affecting application programs—a much stricter requirement.

Whenever we have a multiple-level DBMS, its catalog must be expanded to include information on how to map requests and data among the various levels. The DBMS uses additional software to accomplish these mappings by referring to the mapping information in the catalog. Data independence occurs because when the schema is changed at some level, the schema at the next higher level remains unchanged; only the *mapping* between the two levels is changed. Hence, application programs referring to the higher-level schema need not be changed.

2.4 The Database System Environment

A DBMS is a complex software system. In this section we discuss the types of software components that constitute a DBMS and the types of computer system software with which the DBMS interacts.

2.4.1 DBMS Component Modules

Figure 2.3 illustrates, in a simplified form, the typical DBMS components. The figure is divided into two parts. The top part of the figure refers to the various users of the database environment and their interfaces. The lower part shows the internal modules of the DBMS responsible for storage of data and processing of transactions.

The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the **operating system (OS)**, which schedules disk read/write. Many DBMSs have their own **buffer management** module to schedule disk read/write, because management of buffer storage has a considerable effect on performance. Reducing disk read/write improves performance considerably. A higher-level **stored data manager** module of the DBMS controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.

Let us consider the top part of Figure 2.3 first. It shows interfaces for the DBA staff, casual users who work with interactive interfaces to formulate queries, application programmers who create programs using some host programming languages, and parametric users who do data entry work by supplying parameters to predefined transactions. The DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands.

The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog. The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.

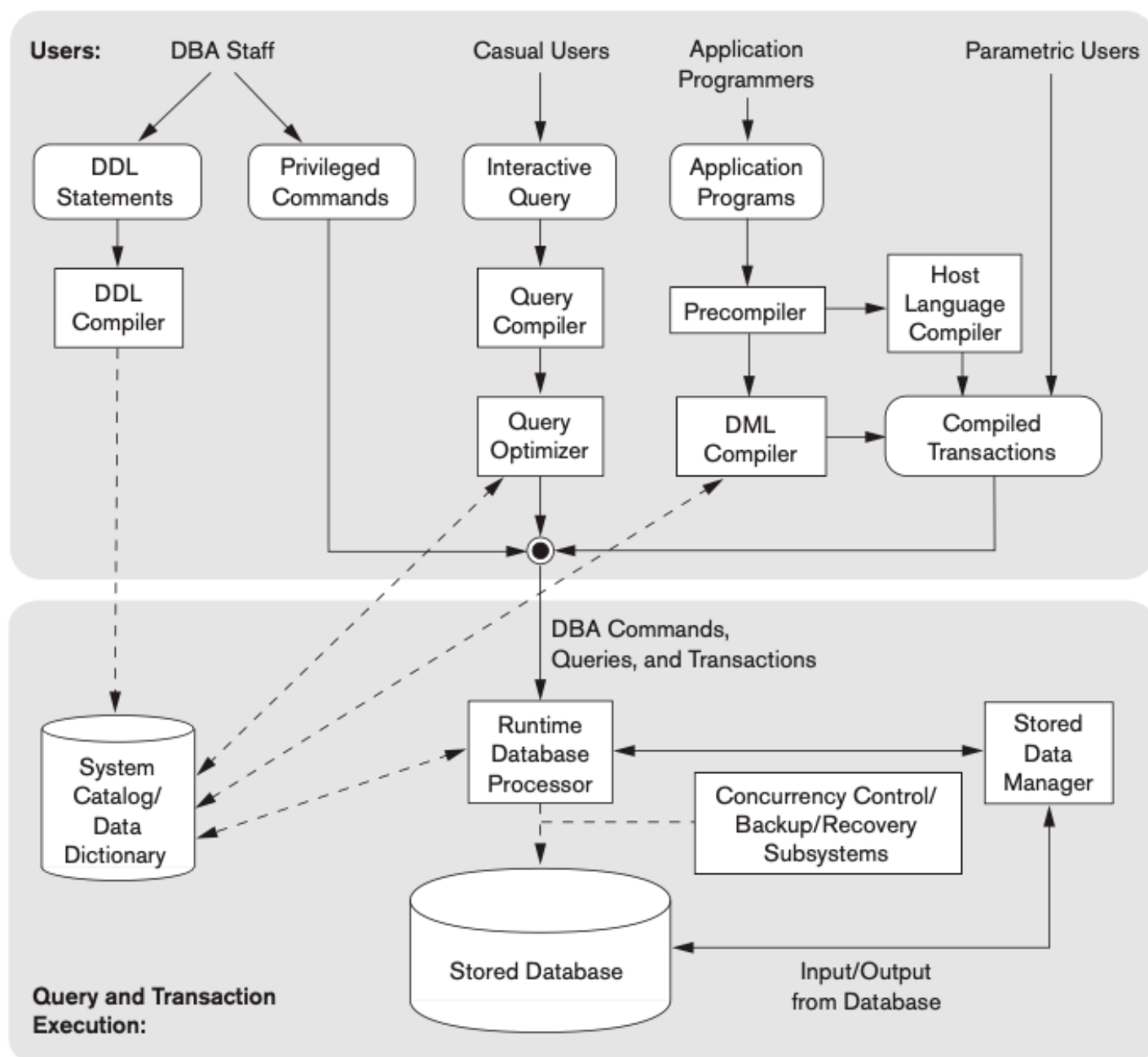


Figure 2.3

Component modules of a DBMS and their interactions.

In addition, the catalog stores many other types of information that are needed by the DBMS modules, which can then look up the catalog information as needed.

Casual users and persons with occasional need for information from the database interact using the **interactive query** interface in Figure 2.3. We have *not explicitly shown* any menu-based or form-based or mobile interactions that are typically used to generate the interactive query automatically or to access canned transactions. These queries are parsed and validated for correctness of the query syntax, the names of files and data elements, and so on by a **query compiler** that compiles

them into an internal form. This internal query is subjected to query optimization (discussed in Chapters 18 and 19). Among other things, the **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of efficient search algorithms during execution. It consults the system catalog for statistical and other physical information about the stored data and generates executable code that performs the necessary operations for the query and makes calls on the runtime processor.

Application programmers write programs in host languages such as Java, C, or C++ that are submitted to a precompiler. The **precompiler** extracts DML commands from an application program written in a host programming language. These commands are sent to the DML compiler for compilation into object code for database access. The rest of the program is sent to the host language compiler. The object codes for the DML commands and the rest of the program are linked, forming a canned transaction whose executable code includes calls to the runtime database processor. It is also becoming increasingly common to use scripting languages such as PHP and Python to write database programs. Canned transactions are executed repeatedly by parametric users via PCs or mobile apps; these users simply supply the parameters to the transactions. Each execution is considered to be a separate transaction. An example is a bank payment transaction where the account number, payee, and amount may be supplied as parameters.

In the lower part of Figure 2.3, the **runtime database processor** executes (1) the privileged commands, (2) the executable query plans, and (3) the canned transactions with runtime parameters. It works with the **system catalog** and may update it with statistics. It also works with the **stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory. The runtime database processor handles other aspects of data transfer, such as management of buffers in the main memory. Some DBMSs have their own buffer management module whereas others depend on the OS for buffer management. We have shown **concurrency control** and **backup and recovery systems** separately as a module in this figure. They are integrated into the working of the runtime database processor for purposes of transaction management.

It is common to have the **client program** that accesses the DBMS running on a separate computer or device from the computer on which the database resides. The former is called the **client computer** running DBMS client software and the latter is called the **database server**. In many cases, the client accesses a middle computer, called the **application server**, which in turn accesses the database server. We elaborate on this topic in Section 2.5.

Figure 2.3 is not meant to describe a specific DBMS; rather, it illustrates typical DBMS modules. The DBMS interacts with the operating system when disk accesses—to the database or to the catalog—are needed. If the computer system is shared by many users, the OS will schedule DBMS disk access requests and DBMS processing along with other processes. On the other hand, if the computer system is mainly dedicated to running the database server, the DBMS will control main memory

buffering of disk pages. The DBMS also interfaces with compilers for general-purpose host programming languages, and with application servers and client programs running on separate machines through the system network interface.

2.4.2 Database System Utilities

In addition to possessing the software modules just described, most DBMSs have **database utilities** that help the DBA manage the database system. Common utilities have the following types of functions:

- **Loading.** A loading utility is used to load existing data files—such as text files or sequential files—into the database. Usually, the current (source) format of the data file and the desired (target) database file structure are specified to the utility, which then automatically reformats the data and stores it in the database. With the proliferation of DBMSs, transferring data from one DBMS to another is becoming common in many organizations. Some vendors offer **conversion tools** that generate the appropriate loading programs, given the existing source and target database storage descriptions (internal schemas).
- **Backup.** A backup utility creates a backup copy of the database, usually by dumping the entire database onto tape or other mass storage medium. The backup copy can be used to restore the database in case of catastrophic disk failure. Incremental backups are also often used, where only changes since the previous backup are recorded. Incremental backup is more complex, but saves storage space.
- **Database storage reorganization.** This utility can be used to reorganize a set of database files into different file organizations and create new access paths to improve performance.
- **Performance monitoring.** Such a utility monitors database usage and provides statistics to the DBA. The DBA uses the statistics in making decisions such as whether or not to reorganize files or whether to add or drop indexes to improve performance.

Other utilities may be available for sorting files, handling data compression, monitoring access by users, interfacing with the network, and performing other functions.

2.4.3 Tools, Application Environments, and Communications Facilities

Other tools are often available to database designers, users, and the DBMS. CASE tools¹² are used in the design phase of database systems. Another tool that can be quite useful in large organizations is an expanded **data dictionary** (or **data repository**)

¹²Although CASE stands for computer-aided software engineering, many CASE tools are used primarily for database design.

system. In addition to storing catalog information about schemas and constraints, the data dictionary stores other information, such as design decisions, usage standards, application program descriptions, and user information. Such a system is also called an **information repository**. This information can be accessed *directly* by users or the DBA when needed. A data dictionary utility is similar to the DBMS catalog, but it includes a wider variety of information and is accessed mainly by users rather than by the DBMS software.

Application development environments, such as PowerBuilder (Sybase) or JBuilder (Borland), have been quite popular. These systems provide an environment for developing database applications and include facilities that help in many facets of database systems, including database design, GUI development, querying and updating, and application program development.

The DBMS also needs to interface with **communications software**, whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or personal computers. These are connected to the database site through data communications hardware such as Internet routers, phone lines, long-haul networks, local networks, or satellite communication devices. Many commercial database systems have communication packages that work with the DBMS. The integrated DBMS and data communications system is called a **DB/DC system**. In addition, some distributed DBMSs are physically distributed over multiple machines. In this case, communications networks are needed to connect the machines. These are often **local area networks (LANs)**, but they can also be other types of networks.

USING HIGH LEVEL CONCEPTUAL DATA MODELS FOR DATABASE DESIGN:

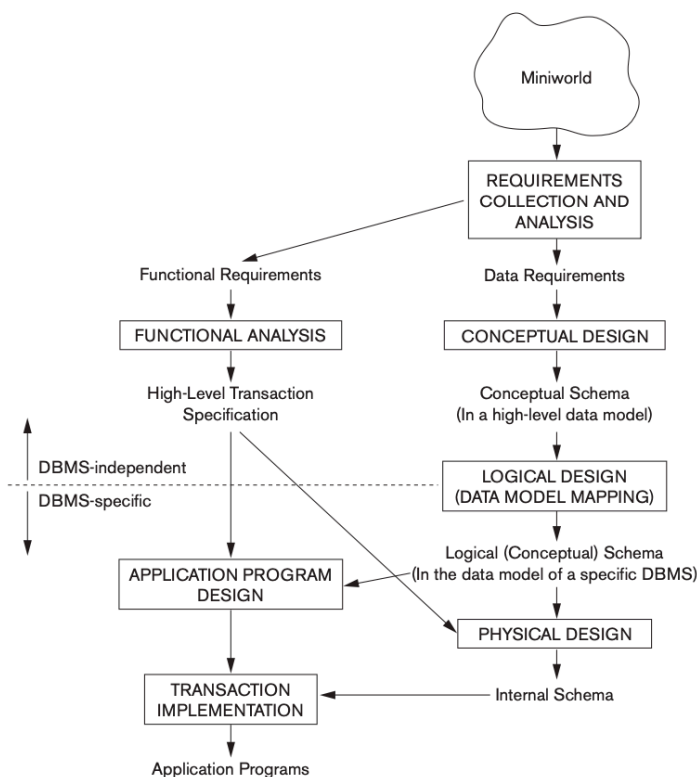


Figure 3.1
A simplified diagram to illustrate the main phases of database design.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS:

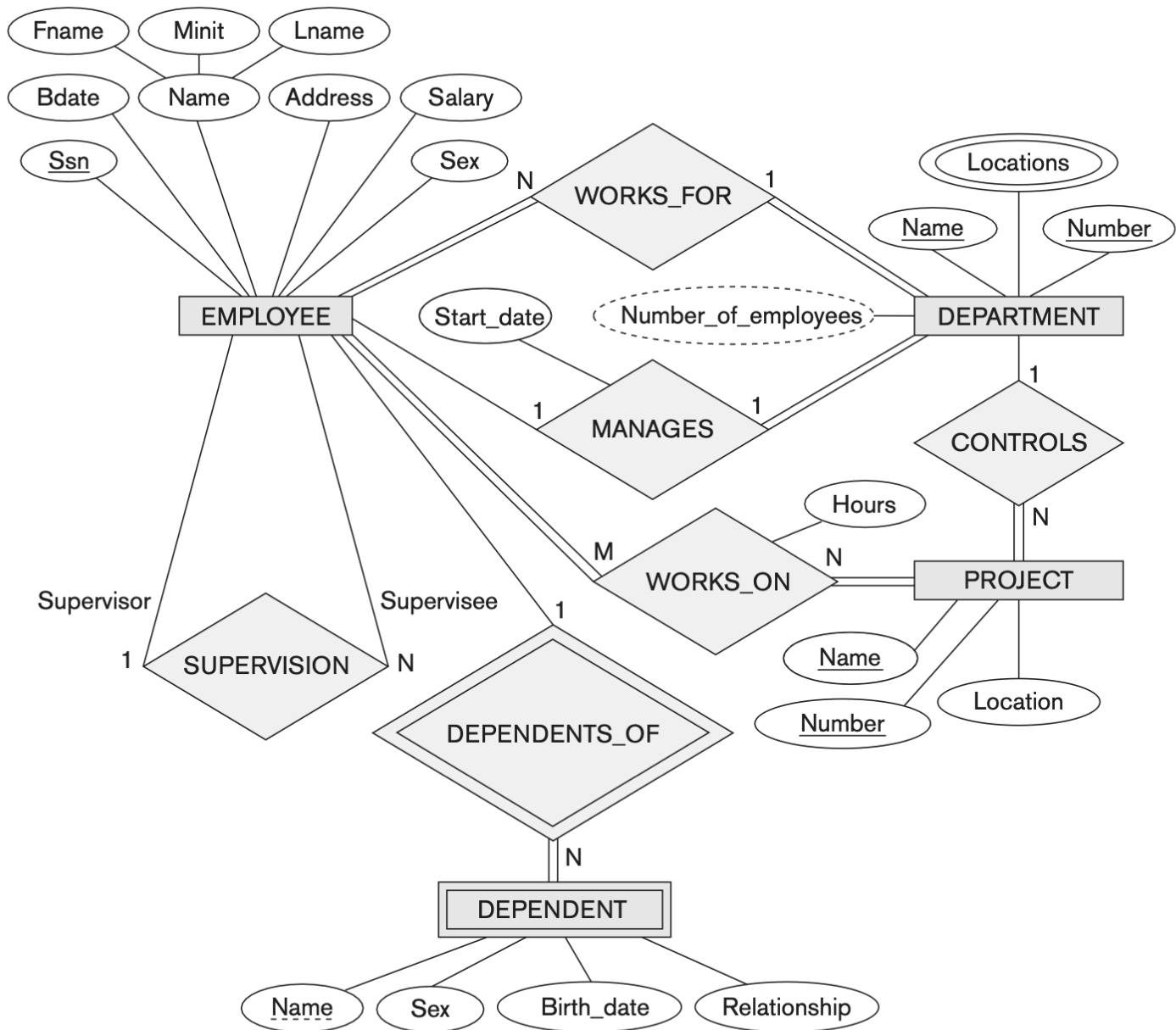


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 3.14.

⇒ Entities and attributes:

- an entity is a thing or object in the real world with an independent existence.
- an entity may be an object with conceptual existence

- each entity has attributes - the particular properties that describe it.
- a particular entity will have a value for each of its attributes
- the attribute values that describe each entity become a major part of the data stored in the database.

→ Types of attributes:

(i) Composite vs Simple (Atomic) Attributes:

- Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings.
- attributes that are not divisible are called simple or atomic attributes.
- composite attributes can form hierarchy.

(ii) Single-valued vs Multivalued attributes:

- Most attributes have a single value for a particular entity.
 - ↳ single-valued.
- multivalued attributes are those that have diff no. of values
 - ↳ there may be upper and lower bound.

(iii) Stored vs Derived attributes:

eg: age is a derived attribute from the stored attribute DOB.

(iv) NULL values :

- ↳ some attributes may not have an applicable value
- ↳ NULL

(v) Complex Attributes :

Complex Attributes. Notice that, in general, composite and multivalued attributes can be nested arbitrarily. We can represent arbitrary nesting by grouping

```
{Address_phone( {Phone(Area_code,Phone_number)},Address(Street_address  
(Number,Street,Apartment_number),City,State,Zip) )}
```

Figure 3.5
A complex attribute:
Address_phone.

components of a composite attribute between parentheses () and separating the components with commas, and by displaying multivalued attributes between braces { }. Such attributes are called **complex attributes**. For example, if a person can have more than one residence and each residence can have a single address and multiple phones, an attribute Address_phone for a person can be specified as shown in Figure 3.5.⁴ Both Phone and Address are themselves composite attributes.

⇒ Entity types and Entity Set

- ↳ a database usually contains groups of entities that are similar.
- ↳ An entity type defines a collection (or set) of entities that have the same attributes
- ↳ each entity type in the database is described by its name and attributes.

- ↳ the collection of all entities of a particular entity type in the database at any point in time is called an entity set or entity collection.
- ↳ an entity type is represented in ER diagrams as a rectangular box enclosing the entity type name.
- ↳ attribute names are enclosed in ovals and are attached to their entity types by straight lines
- ↳ multivalued attributes are displayed in double ovals.
- ↳ an entity type describes the schema or intension for a set of entities that share the same structure.
- ↳ the collection of entities of a particular entity type is grouped into an entity set, which is also called the extension of the entity type.

Key Attributes of an Entity Type. An important constraint on the entities of an entity type is the **key** or **uniqueness constraint** on attributes. An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely. For example, the Name attribute is a key of the COMPANY entity type in Figure 3.6 because no two companies are allowed to have the same name. For the PERSON entity type, a typical key attribute is Ssn (Social Security number). Sometimes several attributes together form a key, meaning that the *combination* of the attribute values must be distinct for each entity. If a set of attributes possesses this property, the proper way to represent this in the ER model that we describe here is to define a *composite attribute* and designate it as a key attribute of the entity type. Notice that such a composite key must be *minimal*; that is, all component attributes must be included in the composite attribute to have the uniqueness property. Superfluous attributes must not be included in a key. In ER diagrammatic notation, each key attribute has its name **underlined** inside the oval, as illustrated in Figure 3.7(a).

Mathematically, an attribute A of entity set E whose value set is V can be defined as a function from E to the power set $P(V)$ of V :

$$A: E \rightarrow P(V)$$

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, & STRUCTURAL CONSTRAINTS:

⇒ Relationship Types, Sets, and Instances:

- ↳ a relationship type R among n entity types $E_1, E_2, \dots, E_n$ defines a set of associations – or a relationship set – among entities from these entity types.
- Mathematically, the relationship set R is a set of relationship instances r_i , where each r_i associates n individual entities $(e_1, e_2, \dots, e_n)$ and each entity e_j in r_i is a member of entity set E_j , $1 \leq j \leq n$.
- Hence, a relationship set is a mathematical relation on $E_1, E_2, \dots, E_n$.

3.4.2 Relationship Degree, Role Names, and Recursive Relationships

Degree of a Relationship Type. The degree of a relationship type is the number of participating entity types. Hence, the WORKS_FOR relationship is of degree two. A relationship type of degree two is called **binary**, and one of degree three is called **ternary**. An example of a ternary relationship is SUPPLY, shown in Figure 3.10, where each relationship instance r_i associates three entities—a supplier s , a part p , and a project j —whenever s supplies part p to project j . Relationships can

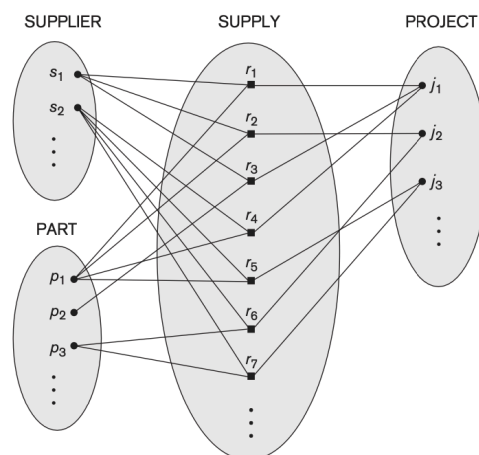


Figure 3.10
Some relationship instances in the SUPPLY ternary relationship set.

Relationships as Attributes. It is sometimes convenient to think of a binary relationship type in terms of attributes, as we discussed in Section 3.3.3. Consider the WORKS_FOR relationship type in Figure 3.9. One can think of an attribute called Department of the EMPLOYEE entity type, where the value of Department for each EMPLOYEE entity is (a reference to) the DEPARTMENT entity for which that employee works. Hence, the value set for this Department attribute is the set of *all* DEPARTMENT entities, which is the DEPARTMENT entity set. This is what we did in Figure 3.8 when we specified the initial design of the entity type EMPLOYEE for the COMPANY database. However, when we think of a binary relationship as an attribute, we always have two options or two points of view. In this example, the alternative point of view is to think of a multivalued attribute Employees of the entity type DEPARTMENT whose value for each DEPARTMENT entity is the set of EMPLOYEE entities who work for that department. The value set of this Employees attribute is the power set of the EMPLOYEE entity set. Either of these two attributes—Department of EMPLOYEE or Employees of DEPARTMENT—can represent the WORKS_FOR relationship type. If both are represented, they are constrained to be inverses of each other.⁸

⁸This concept of representing relationship types as attributes is used in a class of data models called **functional data models**. In object databases (see Chapter 12), relationships can be represented by reference attributes, either in one direction or in both directions as inverses. In relational databases (see Chapter 5), foreign keys are a type of reference attribute used to represent relationships.

Role Names and Recursive Relationships. Each entity type that participates in a relationship type plays a particular role in the relationship. The **role name** signifies the role that a participating entity from the entity type plays in each relationship instance, and it helps to explain what the relationship means. For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of *employee or worker* and DEPARTMENT plays the role of *department or employer*.

Role names are not technically necessary in relationship types where all the participating entity types are distinct, since each participating entity type name can be used as the role name. However, in some cases the *same* entity type participates more than once in a relationship type in *different roles*. In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays. Such relationship types are called **recursive relationships** or **self-referencing relationships**. Figure 3.11 shows an example. The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set. Hence, the EMPLOYEE entity type *participates twice* in SUPERVISION: once in the role of *supervisor* (or *boss*), and once in the role of *supervisee* (or *subordinate*). Each relationship instance r_i in SUPERVISION associates two different employee entities e_j and e_k , one of which plays the role of supervisor and the other the role of supervisee. In Figure 3.11, the lines marked ‘1’ represent the supervisor role, and those marked ‘2’ represent the supervisee role; hence, e_1 supervises e_2 and e_3 , e_4 supervises e_6 and e_7 , and e_5 supervises e_1 and e_4 . In this example, each relationship instance must be connected with two lines, one marked with ‘1’ (supervisor) and the other with ‘2’ (supervisee).

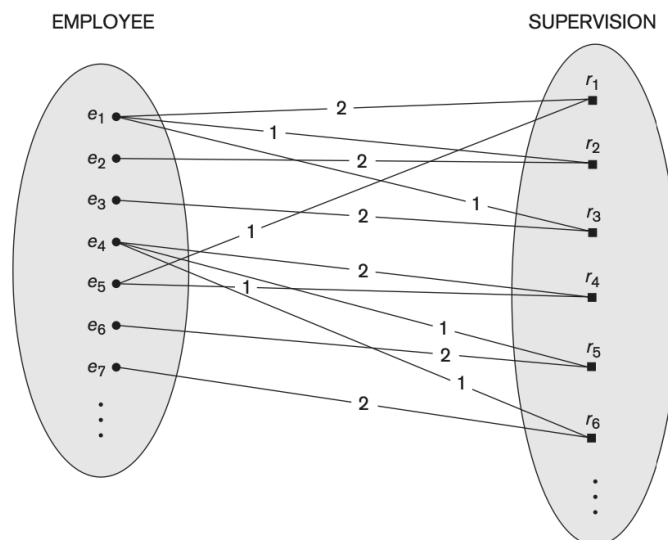


Figure 3.11
A recursive relationship SUPERVISION between EMPLOYEE in the supervisor role (1) and EMPLOYEE in the subordinate role (2).

3.4.4 Attributes of Relationship Types

Relationship types can also have attributes, similar to those of entity types. For example, to record the number of hours per week that a particular employee works on a particular project, we can include an attribute `Hours` for the `WORKS_ON` relationship type in Figure 3.13. Another example is to include the date on which a manager started managing a department via an attribute `Start_date` for the `MANAGES` relationship type in Figure 3.12.

Notice that attributes of 1:1 or 1:N relationship types can be migrated to one of the participating entity types. For example, the `Start_date` attribute for the `MANAGES` relationship can be an attribute of either `EMPLOYEE` (manager) or `DEPARTMENT`, although conceptually it belongs to `MANAGES`. This is because `MANAGES` is a 1:1 relationship, so every department or employee entity participates in *at most one* relationship instance. Hence, the value of the `Start_date` attribute can be determined separately, either by the participating department entity or by the participating employee (manager) entity.

For a 1:N relationship type, a relationship attribute can be migrated *only* to the entity type on the N-side of the relationship. For example, in Figure 3.9, if the `WORKS_FOR` relationship also has an attribute `Start_date` that indicates when an employee started working for a department, this attribute can be included as an attribute of `EMPLOYEE`. This is because each employee works for at most one department, and hence participates in at most one relationship instance in `WORKS_FOR`, but a department can have many employees, each with a different start date. In both 1:1 and 1:N relationship types, the decision where to place a relationship attribute—as a relationship type attribute or as an attribute of a participating entity type—is determined subjectively by the schema designer.

For M:N (many-to-many) relationship types, some attributes may be determined by the *combination of participating entities* in a relationship instance, not by any single entity. Such attributes *must be specified as relationship attributes*. An example is the `Hours` attribute of the M:N relationship `WORKS_ON` (Figure 3.13); the number of hours per week an employee currently works on a project is determined by an employee-project combination and not separately by either entity.

3.5 Weak Entity Types

Entity types that do not have key attributes of their own are called **weak entity types**. In contrast, **regular entity types** that do have a key attribute—which include all the examples discussed so far—are called **strong entity types**. Entities belonging to a weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values. We call this other entity type the **identifying or owner entity type**,¹⁰ and we call the relationship type that relates a weak entity type to its owner the **identifying relationship** of the weak entity type.¹¹ A weak entity type always has a *total participation constraint* (existence dependency) with respect to its identifying relationship because a weak entity cannot be identified without an owner entity. However, not every existence dependency results in a weak entity type. For example, a DRIVER_LICENSE entity cannot exist unless it is related to a PERSON entity, even though it has its own key (License_number) and hence is not a weak entity.

Consider the entity type DEPENDENT, related to EMPLOYEE, which is used to keep track of the dependents of each employee via a 1:N relationship (Figure 3.2). In our example, the attributes of DEPENDENT are Name (the first name of the dependent), Birth_date, Sex, and Relationship (to the employee). Two dependents of *two distinct employees* may, by chance, have the same values for Name, Birth_date, Sex, and Relationship, but they are still distinct entities. They are identified as distinct entities only after determining the *particular employee entity* to which each dependent is related. Each employee entity is said to *own* the dependent entities that are related to it.

A weak entity type normally has a **partial key**, which is the attribute that can uniquely identify weak entities that are *related to the same owner entity*.¹² In our example, if we assume that no two dependents of the same employee ever have the same first name, the attribute Name of DEPENDENT is the partial key. In the worst case, a composite attribute of *all the weak entity's attributes* will be the partial key.

In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines (see Figure 3.2). The partial key attribute is underlined with a dashed or dotted line.

Weak entity types can sometimes be represented as complex (composite, multivalued) attributes. In the preceding example, we could specify a multivalued attribute Dependents for EMPLOYEE, which is a multivalued composite attribute with the component attributes Name, Birth_date, Sex, and Relationship. The choice of which representation to use is made by the database designer. One criterion that may be used is to choose the weak entity type representation if the weak entity type participates independently in relationship types other than its identifying relationship type.

In general, any number of levels of weak entity types can be defined; an owner entity type may itself be a weak entity type. In addition, a weak entity type may have more than one identifying entity type and an identifying relationship type of degree higher than two, as we illustrate in Section 3.9.

¹⁰The identifying entity type is also sometimes called the **parent entity type** or the **dominant entity type**.

¹¹The weak entity type is also sometimes called the **child entity type** or the **subordinate entity type**.

¹²The partial key is sometimes called the **discriminator**.